PostgreSQL Basics, Architecture and SQL Language Commands

Pooja T S, Faculty of Computer Applications, RR Campus, PES University

February 2025

1 Introduction

PostgreSQL is an advanced, open-source, object–relational database system that extends SQL with support for complex data types, JSON, and robust transaction handling. It is known for performance, stability, and compliance with the ACID model for reliable data storage.

2 Overview of PostgreSQL

2.1 What is PostgreSQL?

PostgreSQL is an open-source object–relational database management system (ORDBMS) that enhances SQL with extensibility, indexing, and advanced concurrency handling.

2.2 Brief History

- Developed from the **POSTGRES** project at UC Berkeley (1986) by Professor Michael Stonebraker.
- Released as open-source **PostgreSQL 6.0** in 1997.
- Modern versions (PostgreSQL 17.x) add JSON, parallelism, and replication support.

2.3 Key Features

Feature	**Description**
ACID Compliance	Guarantees Atomicity, Consistency, Isolation, and Durability.
MVCC (Multi-Version Concurrency Control)	Enables multiple users to read/write simultaneously without blocking.
Extensibility	Custom data types, operators, and functions supported.
Advanced Indexing	Offers B-Tree, Hash, GIN, GiST, BRIN indexes.
JSON and JSONB Support	Stores and queries semi-structured data; JSONB supports indexing.
Full-text Search	In-built full-text search using `tsvector` and `tsquery`.
Replication and Backup	Streaming replication, point-in-time recovery, WAL archiving.
Security and Roles	Role-based authentication, SSL, privilege management.
Cross-Platform	Works on Windows, macOS, and Linux.

2.4 Comparison with MySQL and SQL Server

Feature	PostgreSQL	MySQL	SQL Server
Type	Object–Relational DBMS	Relational DBMS	Proprietary RDBMS
License	Open-source (PostgreSQL License)	Open-source (GPL)	Commercial
Standards Compliance	SQL:2016 compliant	Partial	High
Transactions	Fully ACID compliant	Engine dependent	Fully ACID compliant
Concurrency	MVCC (non-blocking reads)	Lock-based	Lock-based
JSON Support	JSON, JSONB with indexing	Basic JSON only	JSON functions
Procedural Language	PL/pgSQL, PL/Python, etc.	Stored procedures	T-SQL
Replication	Streaming + Logical	Master–Slave	AlwaysOn Availability
Best Used For	Analytics, Web, Research, GIS	Web and CMS apps	Enterprise business systems

2.5 Installation Overview

- **Windows:** Download installer from `https://www.enterprisedb.com/downloads/postgres-postgresql-downloads` and install pgAdmin + Server.
- **macOS:** Install via Homebrew: `brew install postgresql@17`; start service: `brew services start postgresql@17`.
- **Linux:** Use package manager:

```
sudo apt install postgresql-17 postgresql-client-17
# or
sudo dnf install postgresql17-server
```

2.6 Accessing PostgreSQL

- **psql Shell:** Interactive terminal to run SQL commands (`psql -U postgres`)
- **pgAdmin 4:** GUI for managing databases and users.
- **PostgreSQL Compass:** Modern GUI for database queries (`localhost:5432`).
- **System Terminal:** Use `psql` commands directly from OS shell.

3 PostgreSQL Architecture

- PostgreSQL server manages multiple databases.
- Each database contains schemas.
- Schemas contain tables, views, sequences, and functions.

Client → psql / pgAdmin → Server → Database → Schema → Tables

4 Writing PostgreSQL Commands

- End every command with a semicolon (;).
- Keywords are case-insensitive but usually written in uppercase.
- Table/column names default to lowercase unless quoted.
- Strings use single quotes (' '), numbers without quotes.

Command Pattern:

```
COMMAND target_object [clauses];
```

Example:

```
CREATE TABLE student (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    marks NUMERIC(5,2),
    city VARCHAR(30)
);
```

5 Categories of SQL Commands

Category	Purpose	Examples
DDL	Define or modify structures	CREATE, ALTER, DROP, TRUNCATE
DML	Manage data	INSERT, UPDATE, DELETE, SELECT
DCL	Control access	GRANT, REVOKE
TCL	Manage transactions	BEGIN, COMMIT, ROLLBACK, SAVEPOINT
Meta	PostgreSQL internal commands	\l, \c, \dt, \d, \q

6 PostgreSQL Data Types (Summary)

Include all numeric, text, date/time, boolean, and special types (JSONB, UUID, ARRAY) for data representation. Refer to detailed table in main text.

7 SQL Language Components

7.1 1. Data Definition Language (DDL)

Purpose: Create or alter database structures.

```
CREATE DATABASE college_db;
\c college_db
CREATE TABLE student (
    sid SERIAL PRIMARY KEY,
    sname VARCHAR(50),
    dept VARCHAR(30),
    marks NUMERIC(5,2),
    city VARCHAR(40)
);
ALTER TABLE student ADD COLUMN email VARCHAR(60);
DROP TABLE student;
```

7.2 2. Data Manipulation Language (DML)

Purpose: Manage table data.

```
INSERT INTO student (sname, dept, marks, city)
VALUES ('Ravi','CSE',88.5,'Bangalore'),
       ('Kavya','ECE',91.2,'Chennai');
SELECT * FROM student WHERE marks$>$80;
UPDATE student SET city='Hyderabad' WHERE sname='Kavya';
DELETE FROM student WHERE dept='ECE';
```

7.3 3. Data Control Language (DCL)

Purpose: Manage user privileges.

```
CREATE USER trainee WITH PASSWORD 'pass123';
GRANT SELECT, INSERT ON student TO trainee;
REVOKE INSERT ON student FROM trainee;
DROP USER trainee;
```

7.4 4. Transaction Control Language (TCL)

Purpose: Manage multiple statements as one transaction.

```
BEGIN;
UPDATE student SET marks=marks+5 WHERE dept='CSE';
DELETE FROM student WHERE marks$<$50;
COMMIT;
-- or rollback
ROLLBACK;
SAVEPOINT sp1;
UPDATE student SET dept='IT' WHERE dept='CSE';
ROLLBACK TO sp1;
```

8 Comparison: Standard SQL vs PostgreSQL

Operation	Standard SQL	PostgreSQL
Create Database	CREATE DATABASE company;	Same
Select Database	USE company;	\c company
List Databases	SHOW DATABASES;	\l
Create Table	CREATE TABLE emp(id INT,name VARCHAR(50));	CREATE TABLE emp(id SERIAL,name VARCHAR(50));
Auto Increment	AUTO_INCREMENT	SERIAL
Describe Table	DESC emp;	\d emp
Show Tables	SHOW TABLES;	\dt
Backup	mysqldump company > backup.sql;	pg_dump company > backup.sql;
Restore	mysql company < file.sql;	psql -d company -f file.sql;
Exit	EXIT;	\q

9 Best Practices

- Write keywords in uppercase for readability.
- Use lowercase for identifiers and underscores instead of spaces.
- End all SQL commands with semicolons.
- Always test DDL/DML commands in a separate environment.
- Use transactions for critical data updates.